Open in app ↗

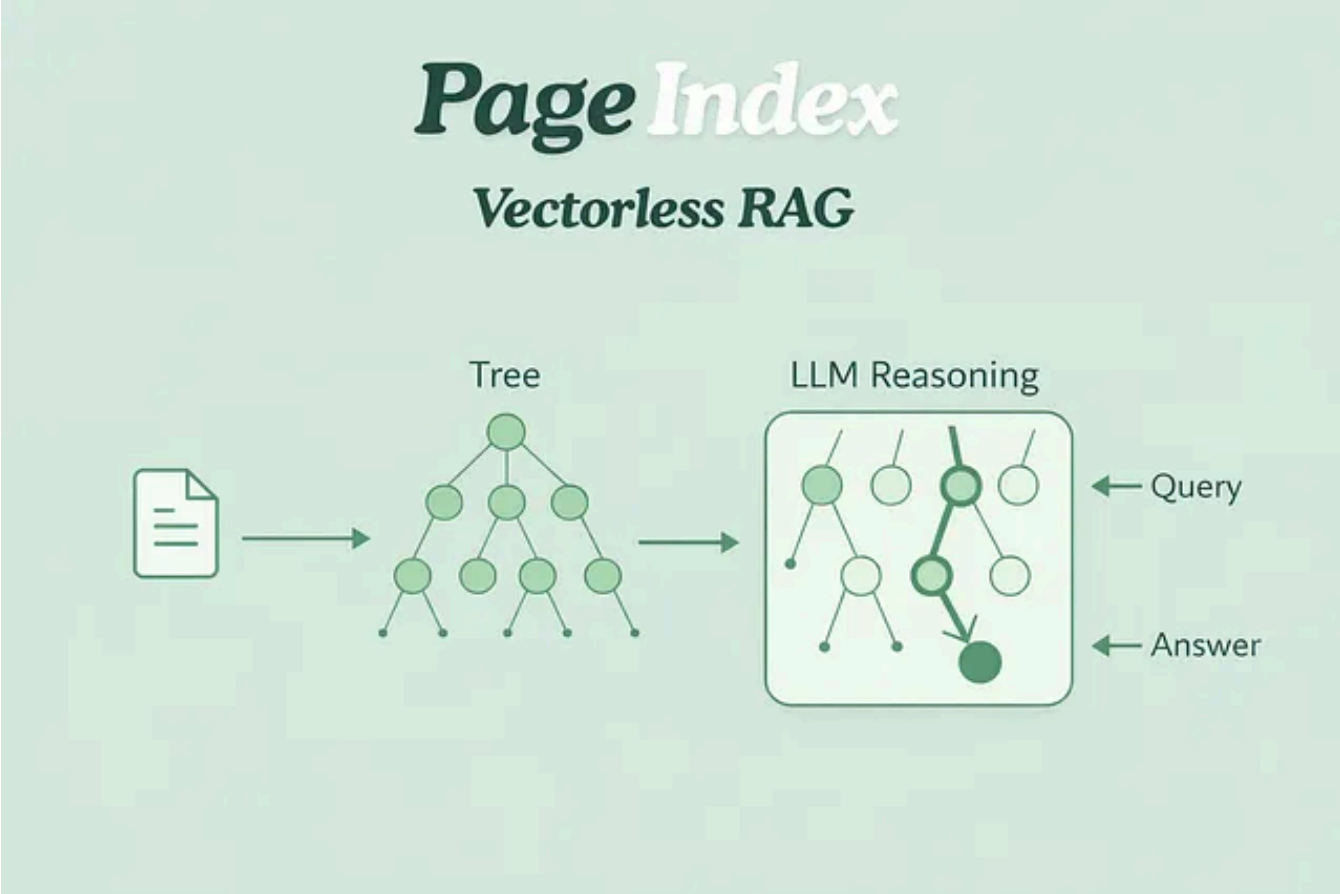
Medium

I Stopped Using Vector Databases for RAG : PageIndex Vectorless RAG

10 min read · Apr 13, 2026

Sweety Tripathi Follow

Listen Share ••• More



If you have been building RAG systems for a while, you know the frustration. You spend hours setting up embeddings, tuning chunk sizes, picking the right vector database, and then your system still returns the wrong answer. Not because it could

not find something similar. But because “similar” and “relevant” are just not the same thing.

If you haven’t explored my RAG series yet, I highly recommend checking it out to better understand the concepts discussed here.

RAG — Complete Tutorial: PART 08

Keyword Search in RAG

medium.com

I ran into this problem when working with financial reports. A 120 page 10-K filing. My vector RAG system kept pulling chunks from the executive summary when the actual answer was buried in a footnote on page 87. Both sections had similar keywords, similar semantic meaning, but only one had the actual number I needed.

That is when I found PageIndex.

What Even Is PageIndex?

PageIndex is an open source RAG framework built by the team at VectifyAI. The core idea is simple but genuinely different from everything else out there. Instead of converting your document into vectors and doing a nearest neighbor search, PageIndex builds a hierarchical tree structure from your document and then uses an LLM to *reason* its way to the right answer.

No embeddings. No vector database. No chunking.

They call it “vectorless RAG” and honestly the name is pretty accurate.

The project was introduced in September 2025 and it gained a lot of attention fast. Mostly because of one benchmark number: **98.7% accuracy on FinanceBench**. For context, traditional vector RAG systems score around 50% on the same benchmark. That is not a small improvement. That is a completely different category of performance.

The Problem with Vector RAG (That Nobody Talks About Enough)

Before we get into how PageIndex works, let us be honest about why vector RAG struggles.

The fundamental assumption of vector RAG is this: the text that is most semantically similar to your query is also the most relevant. That assumption breaks constantly in the real world.

Think about it. If you ask "What was the company's total debt in 2023?" the chunks that come back with high cosine similarity might include:

- The CEO's letter mentioning debt management strategy
- A risk section discussing debt covenants
- A glossary defining long-term debt

All three are semantically close to your query. None of them are the actual answer, which is a specific number in the balance sheet on page 64.

There are a few other issues too.

1. **Chunking destroys context.** When you split a 100-page document into 500-token chunks, you are throwing away the document's natural structure. A section that says "as mentioned in Table 3.2" is completely disconnected from Table 3.2 once you chunk them separately. The document has a logic and a flow. Chunking ignores all of that.
2. **Cross-references are invisible to vectors.** Documents often say things like "see Appendix G" or "refer to the prior year figures in Section 4." Vector similarity has no way to follow those references. It just treats them as text fragments with no connection to where they point.
3. **Queries express intent, not content.** When a user asks a question, they are expressing what they want to know, not what the answer looks like. The answer might use completely different vocabulary than the question. Vector retrieval struggles here because it is matching question text against answer text.

Enter PageIndex- Retrieval by Reasoning

Here is the shift in thinking that makes PageIndex different.

Instead of asking “which chunks look similar to this query?”, PageIndex asks “if I were a human expert, where in this document would I look to answer this question?”

That is a completely different retrieval strategy. And it works much better for structured, complex documents.

How It Actually Works

PageIndex works in two main phases.

Phase 1: Building the Index (Table of Contents Tree)

When you feed a document into PageIndex, it does not chunk it. Instead, it reads the document and builds a hierarchical tree that mirrors the document’s natural structure. Think of it like an intelligent, deeply nested table of contents.

Each node in the tree represents a section or subsection of the document. Each node has a title and a summary that describes what that section contains. The summaries are written by the LLM during indexing, and they become the retrieval signal later.

So for a 10-K filing, the tree might look something like this:

```
Annual Report 2023
├── Business Overview
│   ├── Products and Services
│   └── Market Position
├── Risk Factors
│   ├── Financial Risks
│   └── Operational Risks
├── Financial Statements
│   ├── Balance Sheet
│   │   ├── Assets
│   │   └── Liabilities
│   └── Income Statement
└── Notes to Financial Statements
    ├── Note 1: Accounting Policies
    └── Note 12: Long-term Debt
```

The document is not flat anymore. It has structure. The LLM can navigate this structure the same way a human would flip through a book.

Phase 2: Reasoning-Based Tree Search

When a query comes in, PageIndex does not search a vector database. It gives the LLM the query and the top-level structure of the tree and asks it to reason about where the answer might be.

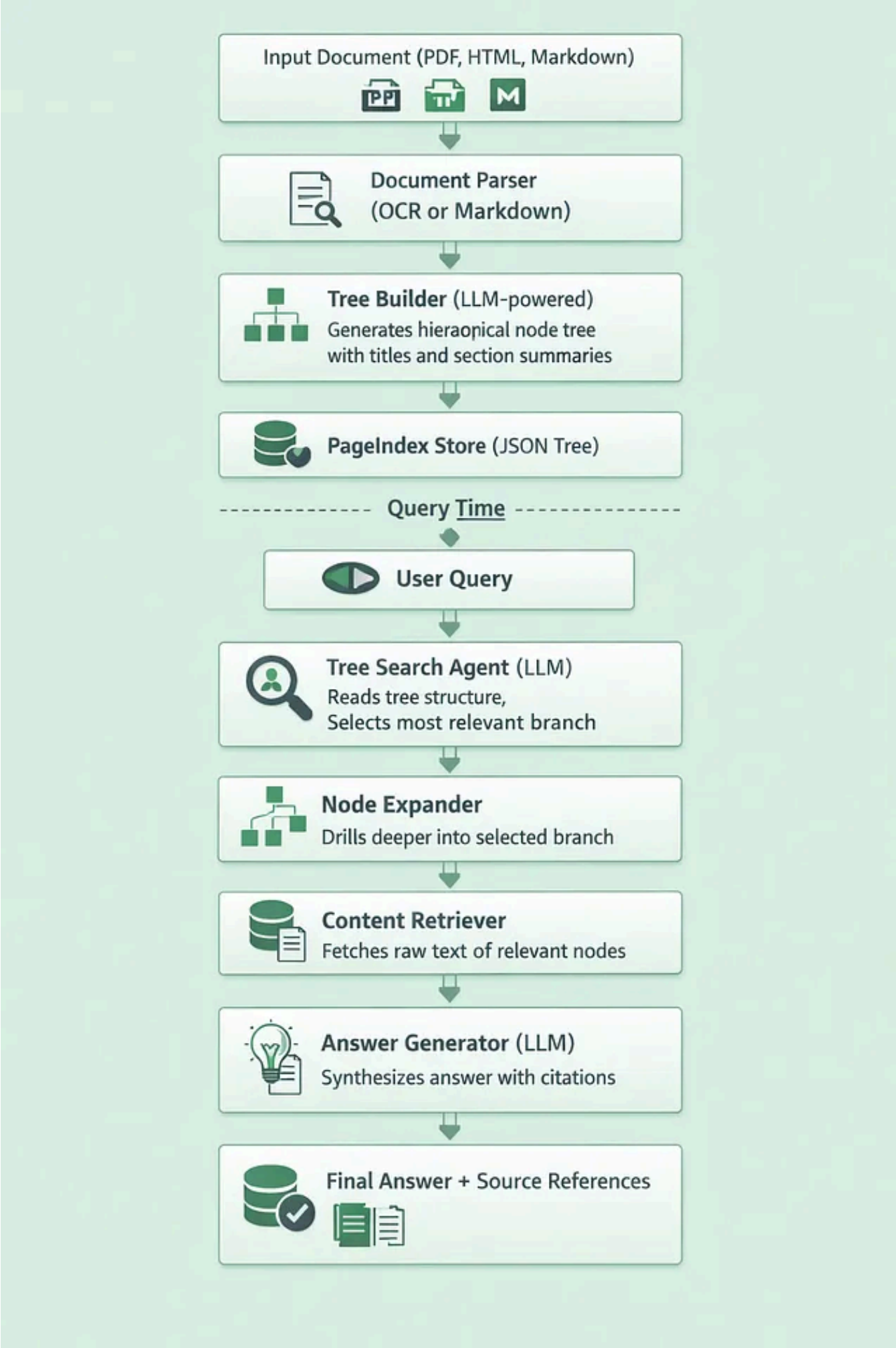
The LLM looks at the tree and says something like: “This question is about long-term debt, so I should look in the Financial Statements section, probably the Notes.” It returns a node ID.

PageIndex then expands that node, showing the LLM the next level of detail. The LLM keeps drilling down until it reaches the actual content that answers the question.

This is how a human analyst works. You do not read a 200-page report from start to finish. You check the table of contents, find the right section, and go straight there. PageIndex is doing exactly that, but at machine speed.

Architecture of PageIndex

Let us look at the architecture a bit more clearly.


Input Document (PDF, HTML, Markdown)
Document Parser
(OCR or Markdown)
Tree Builder (LLM-powered)
Generates hieraonical node tree
with titles and section summaries
PageIndex Store (JSON Tree)
Query Time
User Query
Tree Search Agent (LLM)
Reads tree structure,
Selects most relevant branch
Node Expander
Drills deeper into selected branch
Content Retriever
Fetches raw text of relevant nodes
Answer Generator (LLM)
Synthesizes answer with citations
Final Answer + Source References

The key thing to notice is that every step except the final content fetch is a reasoning step. The LLM is not matching vectors. It is actively thinking about where to look.

Let's Code

Step 1: Parse the Document

Everything starts with extracting raw text from the document, page by page. PyMuPDF handles this cleanly for PDFs.

```
import fitz  # pip install pymupdf

def parse_pdf(pdf_path: str) -> list[dict]:
    doc = fitz.open(pdf_path)
    pages = []
    for i, page in enumerate(doc):
        text = page.get_text().strip()
        if text:
            pages.append({"page_num": i + 1, "text": text})
    doc.close()
    return pages
```

After parsing, we group consecutive pages into sections. This is different from arbitrary chunking because we are respecting natural page boundaries and keeping related content together.

```
def group_pages_into_sections(pages, per_section=3):
    sections = []
    for i in range(0, len(pages), per_section):
        batch = pages[i : i + per_section]
        section_id = f"S{str(i // per_section + 1).zfill(3)}"
        combined_text = "\n\n".join(p["text"] for p in batch)
        sections.append({
            "section_id": section_id,
            "start_page": batch[0]["page_num"],
            "end_page": batch[-1]["page_num"],
            "text": combined_text,
        })
    return sections
```

Step 2: Build the Tree Index (LLM-Powered)

This is the heart of PageIndex. Instead of computing embeddings, we ask Gemini to read a preview of each section and generate a structured summary. The result is a hierarchical JSON tree where every node has a title, a summary, and key topics.

```
from google import genai
from google.genai import types

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")
def index_section(section: dict) -> dict:
    preview = section["text"][:1500]
    prompt = f"""Read this section from a document and summarize it.
Section pages: {section['start_page']} to {section['end_page']}
Text:
{preview}
Respond with ONLY valid JSON:
{{
  "title": "short descriptive title (5-8 words)",
  "summary": "2-3 sentence summary of what this section covers",
  "key_topics": ["topic1", "topic2", "topic3"]
}}"""
    response = client.models.generate_content(
        model="gemini-2.0-flash",
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0.0),
    )
    parsed = json.loads(response.text.strip())
    return {
        "node_id": section["section_id"],
        "title": parsed["title"],
        "pages": f"{section['start_page']}-{section['end_page']}",
        "summary": parsed["summary"],
        "key_topics": parsed["key_topics"],
    }
```

Run this for every section and you get a complete tree stored as a JSON file.

```
def build_tree_index(sections):
    nodes = [index_section(s) for s in sections]
    return {
        "title": "Your Document Title",
        "total_sections": len(nodes),
        "nodes": nodes,
    }
# Save for reuse
```

```
with open("tree.json", "w") as f:
    json.dump(tree, f, indent=2)
```

The tree looks like this in practice:

```
Attention Is All You Need
  9 sections indexed

├── [S001] Abstract and Introduction  (pp. 1-3)
│     Topics: transformer, self-attention, motivation
│     Introduces the Transformer model as a novel architecture based
│     entirely on attention mechanisms, dispensing with recurrence
│     and convolutions.
│
├── [S002] Model Architecture: Encoder and Decoder  (pp. 3-5)
│     Topics: encoder, decoder, stacked layers
│     Describes the encoder-decoder structure. Encoder maps input
│     sequences to continuous representations using stacked
│     self-attention and feed-forward layers.
│
├── [S003] Scaled Dot-Product Attention  (pp. 5-6)
│     Topics: query, key, value, softmax
│     Defines Attention(Q,K,V) = softmax(QK^T / sqrt(d_k))V. Scaling
│     by sqrt(d_k) prevents vanishing gradients in high dimensions.
│
├── [S004] Multi-Head Attention  (pp. 6-7)
│     Topics: multi-head, projection, subspaces
│     Projects queries, keys, and values h times with different
│     learned projections, allowing the model to attend to different
│     representation subspaces.
│
├── [S005] Positional Encoding and Feed-Forward Layers  (pp. 7-8)
│     Topics: positional encoding, sine, cosine, ReLU
│     Sine and cosine positional encodings are added to embeddings
│     since the model has no recurrence. Feed-forward layers apply
│     two linear transforms with ReLU.
│
├── [S006] Training Setup and Regularization  (pp. 8-9)
│     Topics: Adam, dropout, label smoothing, WMT 2014
│     Training on WMT 2014 using Adam optimizer. Dropout applied to
│     sub-layer outputs. Label smoothing used as regularization.
│
├── [S007] Experimental Results and BLEU Scores  (pp. 9-10)
│     Topics: BLEU, WMT 2014, English-German
│     Achieves 28.4 BLEU on EN-DE and 41.0 BLEU on EN-FR, surpassing
│     all previously published models at a fraction of training
│     cost.
│
├── [S008] Model Variations and Ablation Study  (pp. 10-12)
│     Topics: ablation, heads, model size
│     Explores effect of varying number of heads, key/value
│     dimensions, model size, and dropout. Single-head attention is
│     worse than multi-head.
│
└── [S009] Conclusions and Related Work  (pp. 12-15)
      Topics: conclusion, future work, related work
      Concludes that Transformer is the first sequence model based
      entirely on attention. Outlines future directions: images,
      audio, video.
```

Step 3: Tree Search (Reasoning, Not Similarity)

Here is where PageIndex diverges completely from vector RAG. When a question arrives, we do not search a database. We give the LLM the full tree structure and ask

it to reason about which sections are most likely to hold the answer. It thinks like a human analyst flipping through a table of contents.

```
def retrieve_sections(tree: dict, query: str) -> dict:
    # Build a compact text representation of the tree
    tree_text = f"Document: {tree['title']}\n\n"
    for node in tree["nodes"]:
        tree_text += f"[{node['node_id']}] Pages {node['pages']} | {node['title
        tree_text += f"  Summary: {node['summary']}\n"
        tree_text += f"  Topics: {', '.join(node['key_topics'])}\n\n"
        prompt = f"""You are a document retrieval expert.
        Given this document tree, identify which sections most likely answer th
        Think step by step about where a human expert would look.
        {tree_text}
        QUESTION: {query}
        Respond with ONLY valid JSON:
        {{
          "reasoning": "your step-by-step reasoning about where to look",
          "selected_ids": ["S001", "S004"],
          "confidence": "high/medium/low"
        }}"""
            response = client.models.generate_content(
                model="gemini-2.0-flash",
                contents=prompt,
                config=types.GenerateContentConfig(temperature=0.0),
            )
            return json.loads(response.text.strip())
```

The reasoning field is not decoration. It tells you exactly why the model chose those sections. This is what PageIndex means by explainability. You can see the retrieval decision, which is something vector search can never give you.

Step 4: Content Retrieval

Once we know which section IDs to look at, we pull the raw text from those sections. No vector lookup, no embedding computation, just a dictionary lookup on the sections we already parsed.

```
def retrieve_content(selected_ids: list, sections: list) -> str:
    section_map = {s["section_id"]: s for s in sections}
    context_parts = []
    for sid in selected_ids:
            if sid in section_map:
                sec = section_map[sid]
```

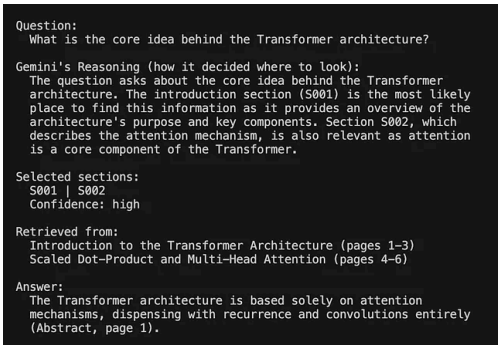
```
                context_parts.append(
                    f"--- Pages {sec['start_page']}-{sec['end_page']} ---\n"
                    + sec["text"][:3000]
                )
        return "\n\n".join(context_parts)
```

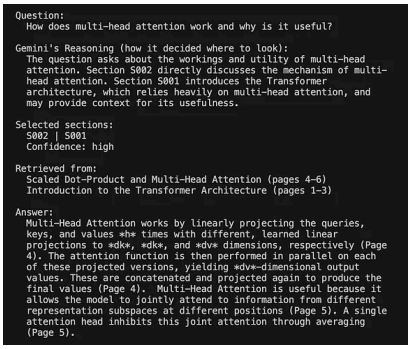
Step 5: Answer Generation

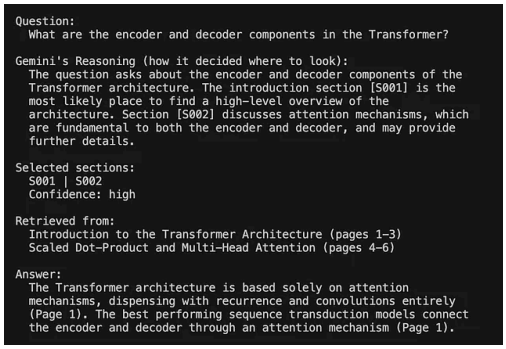
The retrieved text goes to Gemini along with the original question. The model synthesizes a precise answer with page-level citations.

```
def generate_answer(query: str, context: str) -> str:
    prompt = f"""Answer the question using only the provided context.
Be specific. Include exact numbers, technical terms, and cite page numbers.CONT
{context}
QUESTION: {query}
ANSWER:"""
    response = client.models.generate_content(
        model="gemini-2.0-flash",
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0.1),
    )
    return response.text.strip()
```

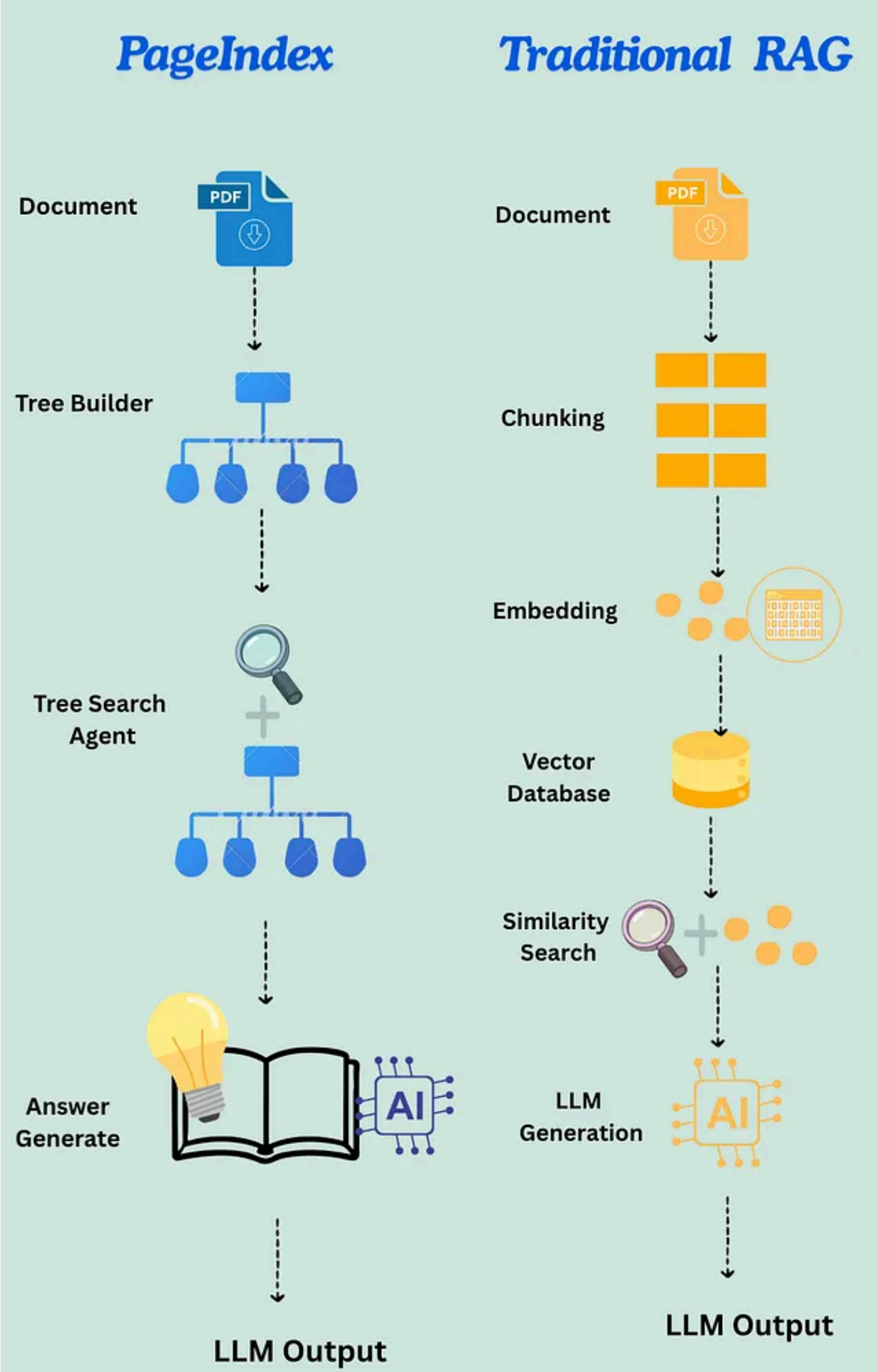
Some Generated Answers 😀:







PageIndex vs Traditional Vector RAG


PageIndex
Traditional RAG
Document
PDF
Tree Builder
Tree Search Agent
Answer Generate
AI
LLM Output
Document
PDF
Chunking
Embedding
Vector Database
Similarity Search
LLM Generation
AI
LLM Output

When Should You Use PageIndex?

This is the most important question. PageIndex is not a replacement for vector RAG in every situation. It is a different tool for a different job.

Use PageIndex when:

You are working with a single long structured document where accuracy matters more than speed. Financial filings, legal contracts, regulatory documents, technical manuals, academic papers. Anything where the document has meaningful hierarchy and where "close enough" is not good enough.

You need explainability. Your users or stakeholders need to know exactly where an answer came from. PageIndex gives you a full reasoning trace with section references and page numbers.

You are building for regulated industries. Finance, legal, healthcare. These are areas where a wrong answer can have real consequences. The 98.7% accuracy number matters here.

Stick with vector RAG when:

You are searching across a large collection of documents and need to find which ones are relevant. Vector RAG is excellent for this. PageIndex is designed to go deep into one document, not to find the right document from a thousand.

You need high throughput and low latency. PageIndex makes multiple sequential LLM calls per query. That adds up in cost and time at scale.

Your documents are short, unstructured, or conversational. PageIndex's strength is navigating document hierarchy. If there is no hierarchy to navigate, you lose the advantage.

Best of both worlds: Use vector search to find the right document from your collection, then use PageIndex to extract the precise answer from within that document. A hybrid approach like this gives you the best of both systems.

Final Thought

What PageIndex is really doing is treating retrieval as a *reasoning problem* instead of a *similarity problem*.

And this shift matters beyond just document retrieval. The same idea is showing up in other places too. Claude Code, for example, has reportedly moved away from vector-based code retrieval in favor of reasoning-driven approaches. The pattern is becoming clearer: when the task requires understanding structure and following logic, reasoning beats similarity.

Vector databases are not going anywhere. They are fast, scalable, and perfect for semantic search across large collections. But for the hard problem of "find me the exact right information inside this complex document," the reasoning-first approach is winning.

PageIndex is one of the best implementations of that idea right now. It is open source, has a clean API, and the benchmark results speak for themselves.

If you are building anything involving dense professional documents, I would strongly recommend trying it. Your users who ask precise questions about 200-page reports will thank you.

Getting Started

You can find the full open source code at github.com/VectifyAI/PageIndex.

The cloud service and API are available at pageindex.ai.

They also have an MCP server if you want to plug PageIndex directly into Claude or other agentic workflows without writing retrieval code from scratch.

Start with the official cookbook examples. They are well written and get you to a working system in about 30 minutes.

Thank you for following this RAG tutorial series! If you found this helpful, please give it a clap 👏 and share with others building such kind of systems.

This story is published on Generative AI. Connect with us on LinkedIn and follow Zeniteq to stay in the loop with the latest AI stories.

Subscribe to our newsletter and YouTube channel to stay updated with the latest news and updates on generative AI. Let's shape the future of AI together!

Vectorless Rag | Vector Database | LLM | AI

Following

Published in Generative AI

84K followers · Last published 1 day ago

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com

Follow

Written by Sweety Tripathi

234 followers · 22 following

Data Scientist | Generative AI Engineer | Writing about how AI learns and reasons. From coding models to decoding intelligence.

No responses yet

Andre Banki

What are your thoughts?